

Problem 1 – Level 1

Scenario:

A small organization wants to store simple log messages into a text file using a C# console application.

Requirements:

1. Accept a message from the user.
2. Write the message into a file using FileStream.
3. Append multiple messages to the same file.
4. Display confirmation after writing the data.

Technical Constraints:

- Use FileStream class.
- Use appropriate FileMode and FileAccess.
- Implement exception handling for file access errors.

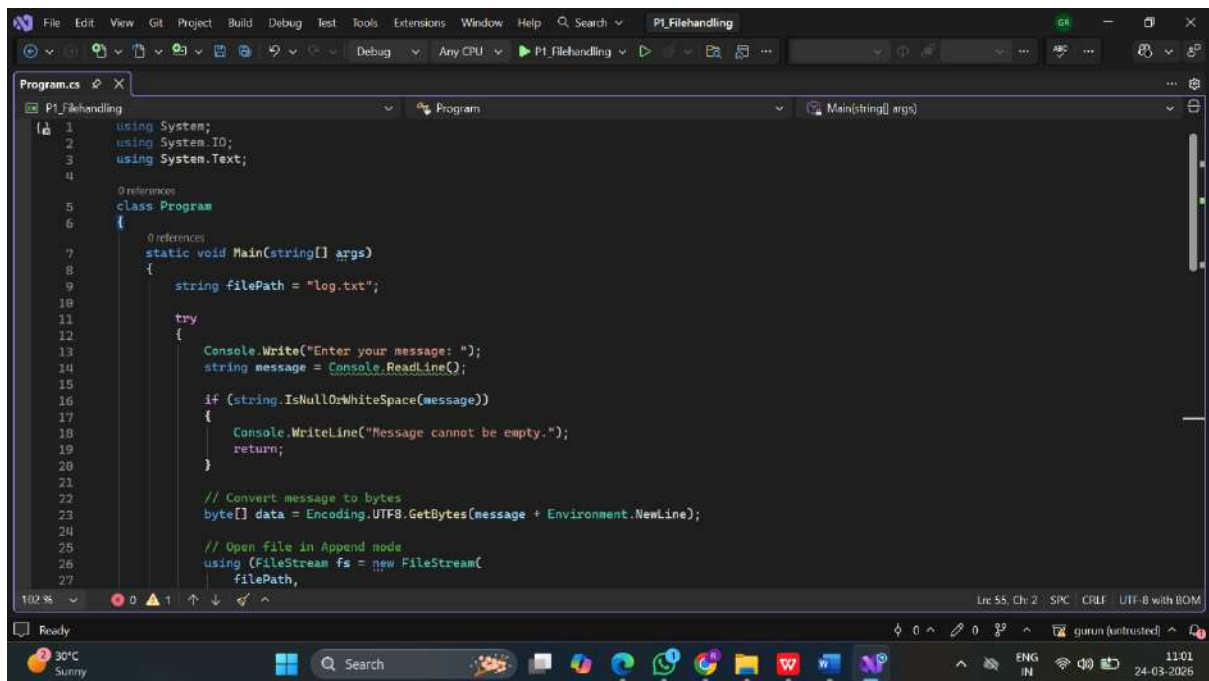
Expectations:

The application should successfully write user messages to the file and allow multiple entries.

Learning Outcome:

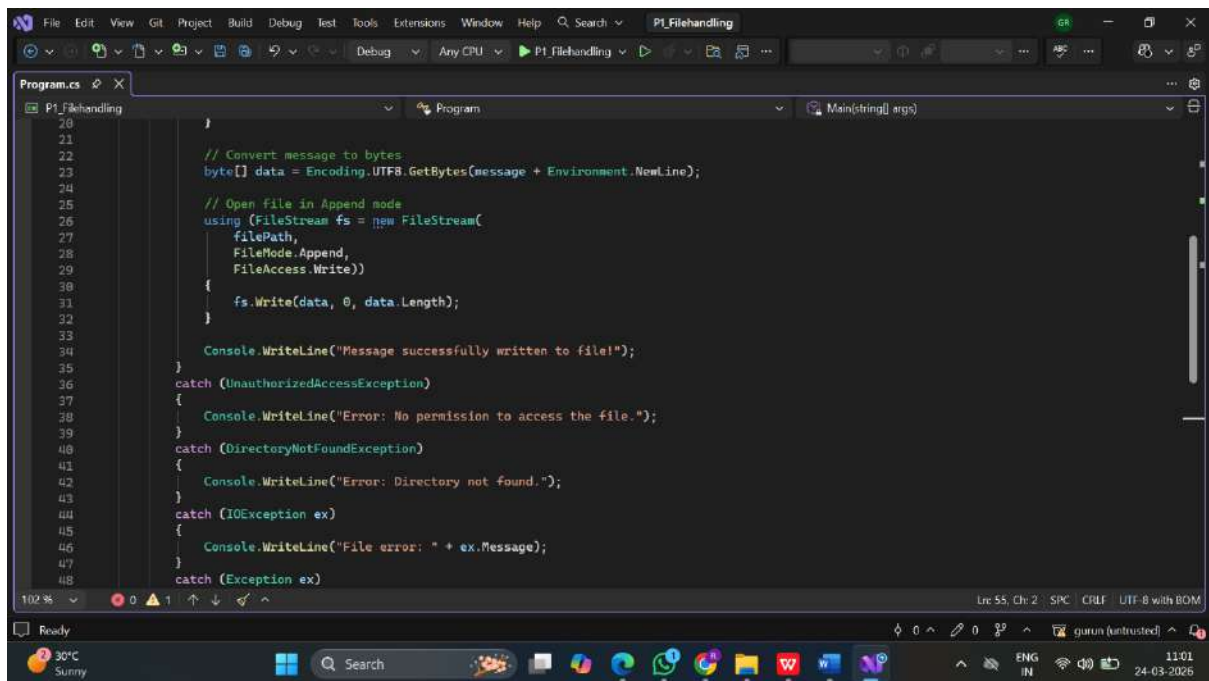
Students will learn how to create and write data into files using FileStream.

Code Screenshots:



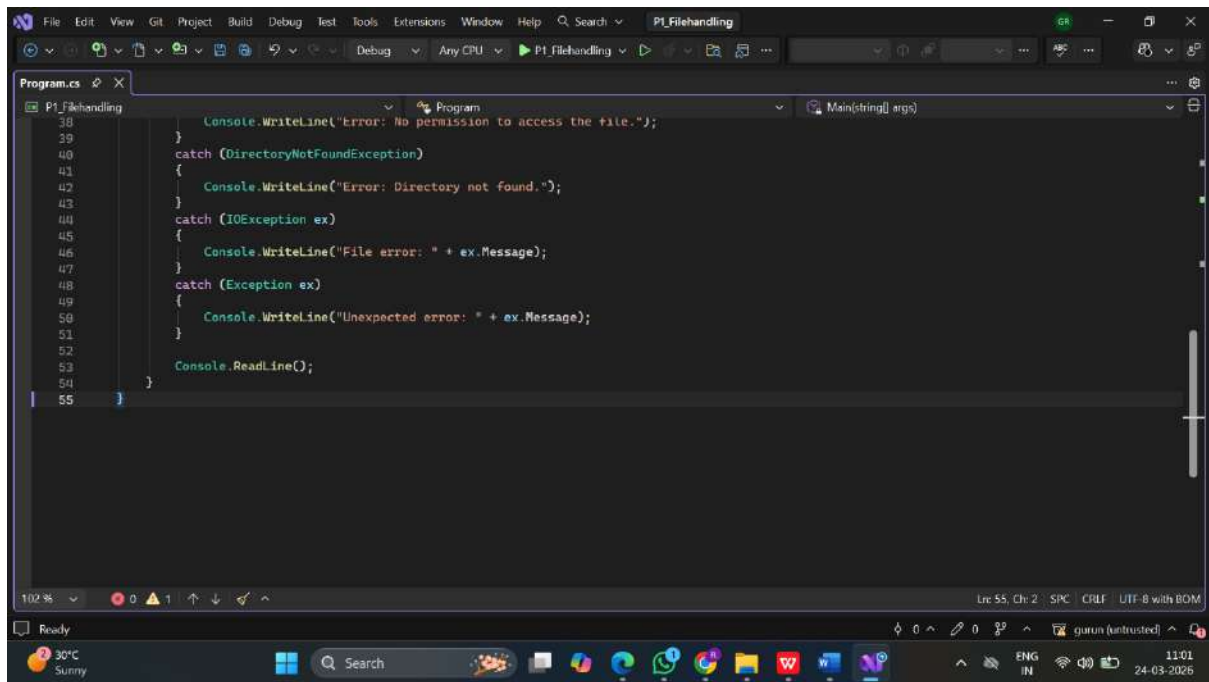
This screenshot shows the first part of a C# program in Visual Studio. The code defines a class `Program` with a static `Main` method. The method prompts the user for a message, checks if it's empty, and then converts the message to bytes. It also opens a file stream in append mode.

```
1 using System;
2 using System.IO;
3 using System.Text;
4
5 // References
6 class Program
7 {
8     // References
9     static void Main(string[] args)
10     {
11         string filePath = "log.txt";
12
13         try
14         {
15             Console.Write("Enter your message: ");
16             string message = Console.ReadLine();
17
18             if (string.IsNullOrEmpty(message))
19             {
20                 Console.WriteLine("Message cannot be empty.");
21                 return;
22             }
23
24             // Convert message to bytes
25             byte[] data = Encoding.UTF8.GetBytes(message + Environment.NewLine);
26
27             // Open file in Append mode
28             using (FileStream fs = new FileStream(
29                 filePath,
```



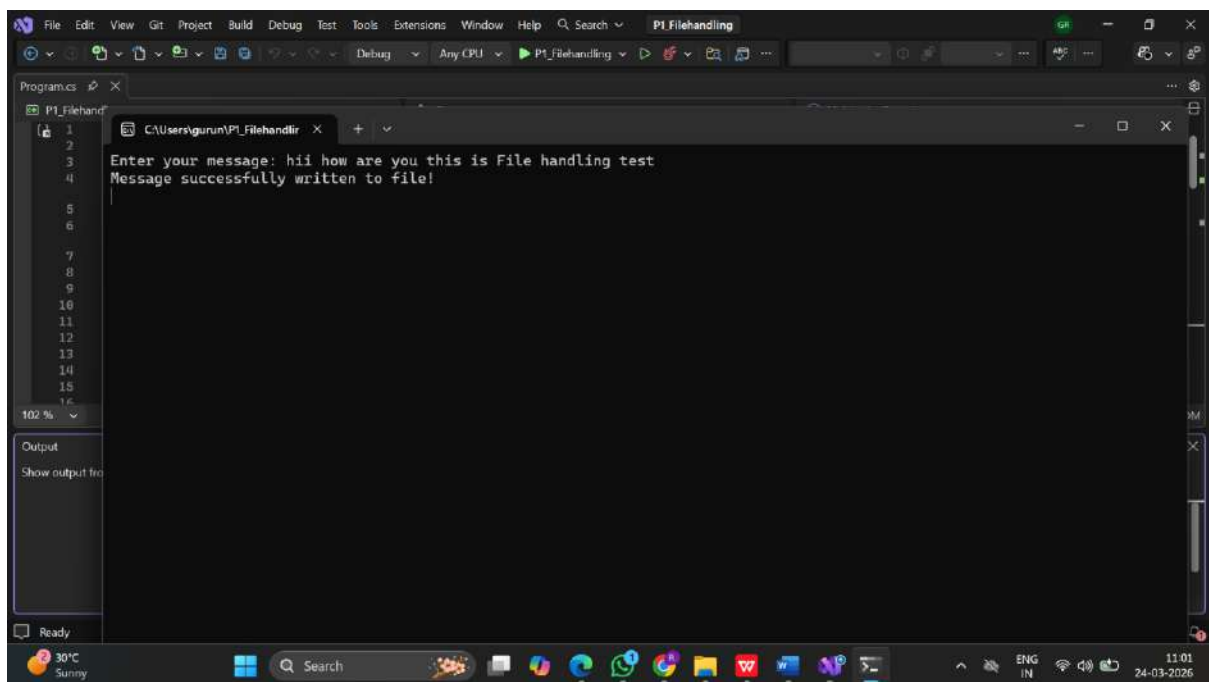
This screenshot shows the second part of the C# program. It continues from the previous code block, showing the file stream being written to, followed by a success message and a series of catch blocks for various exceptions.

```
30         filePath,
31         FileMode.Append,
32         FileAccess.Write))
33         {
34             fs.Write(data, 0, data.Length);
35
36             Console.WriteLine("Message successfully written to file!");
37
38             catch (UnauthorizedAccessException)
39             {
40                 Console.WriteLine("Error: No permission to access the file.");
41             }
42
43             catch (DirectoryNotFoundException)
44             {
45                 Console.WriteLine("Error: Directory not found.");
46             }
47
48             catch (IOException ex)
49             {
50                 Console.WriteLine("File error: " + ex.Message);
51             }
52
53             catch (Exception ex)
54             {
55                 Console.WriteLine("Unexpected error: " + ex.Message);
56             }
57         }
58     }
59 }
```



```
Program.cs
P1_Filehandling
38 Console.WriteLine("Error: No permission to access the file.");
39
40 catch (DirectoryNotFoundException)
41 {
42     Console.WriteLine("Error: Directory not found.");
43 }
44 catch (IOException ex)
45 {
46     Console.WriteLine("File error: " + ex.Message);
47 }
48 catch (Exception ex)
49 {
50     Console.WriteLine("Unexpected error: " + ex.Message);
51 }
52 Console.ReadLine();
53 }
54
55 }
```

Output:



```
Output
C:\Users\gunun\P1_Filehandling
1 Enter your message: hii how are you this is File handling test
2 Message successfully written to file!
3
4
5
6
7
8
9
10
11
12
13
14
15
16
```

Problem 2 – Level 1:

Scenario:

An administrator wants to check file properties stored in a particular folder for auditing purposes.

Requirements:

1. Accept a folder path from the user.
2. Display file name, file size, and creation date.
3. Count and display the total number of files.

Technical Constraints:

- Use FileInfo class.
- Handle invalid directory paths.

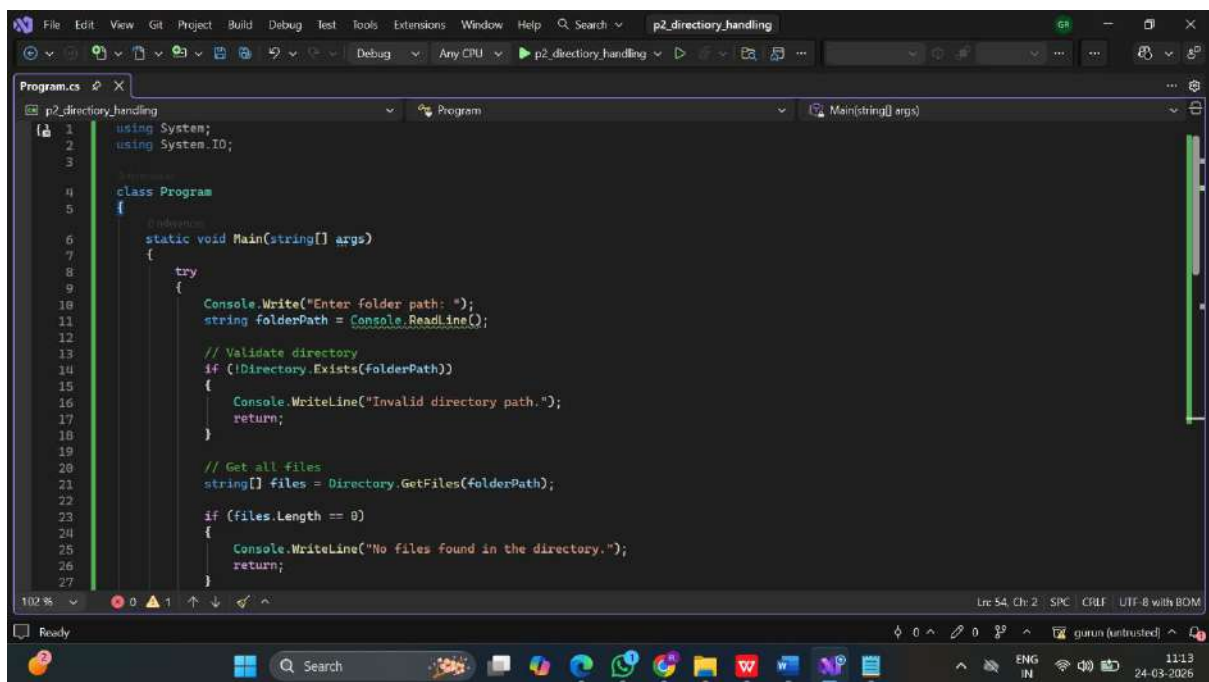
Expectations:

The program should list file details clearly in the console.

Learning Outcome:

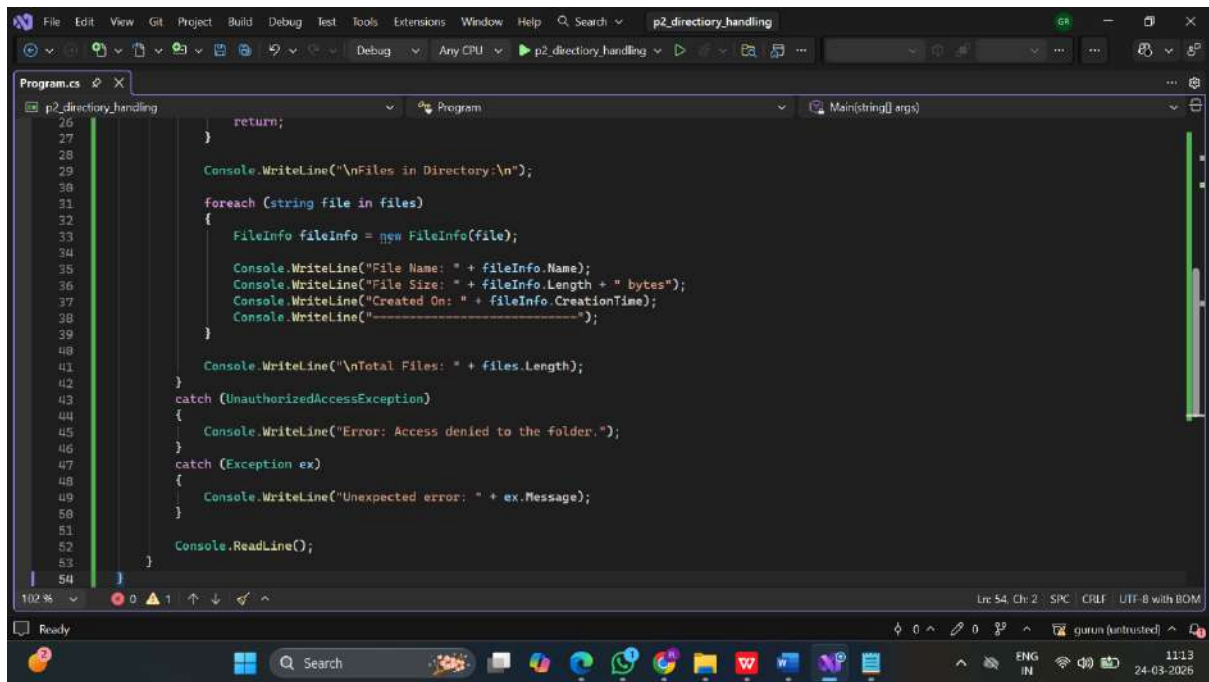
Students will understand how to retrieve file metadata using FileInfo.

Code Screenshot:



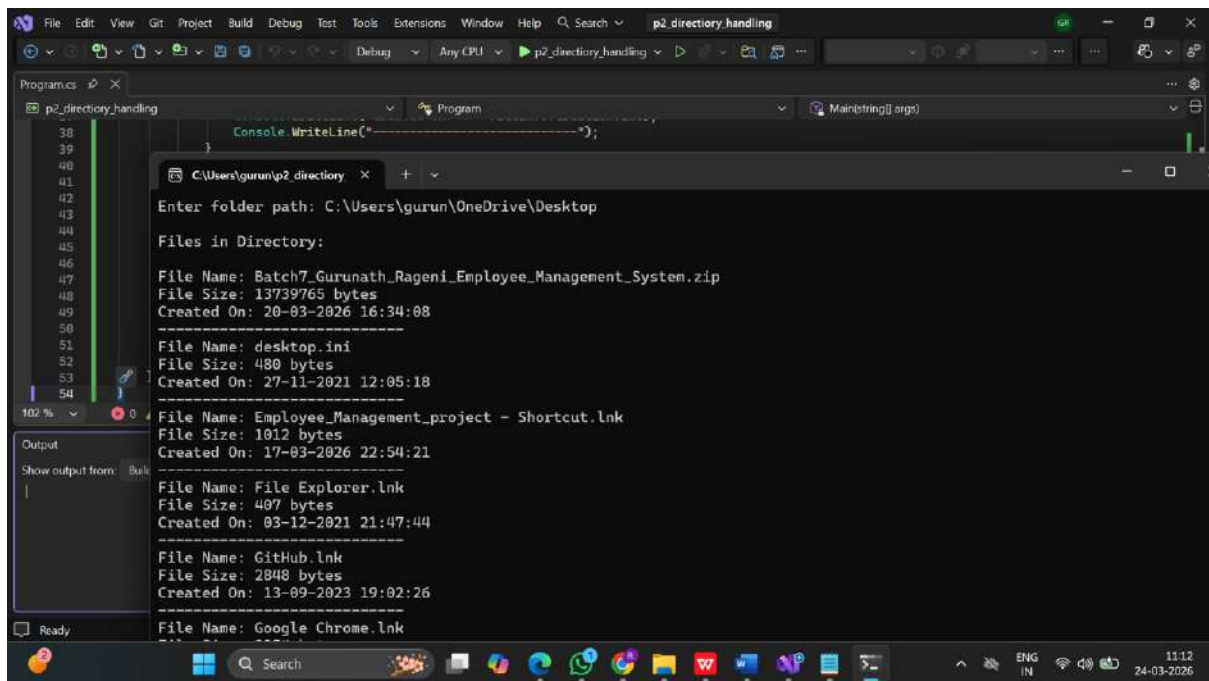
```
Program.cs
1  using System;
2  using System.IO;
3
4  class Program
5  {
6      static void Main(string[] args)
7      {
8          try
9          {
10             Console.WriteLine("Enter folder path: ");
11             string folderPath = Console.ReadLine();
12
13             // Validate directory
14             if (!Directory.Exists(folderPath))
15             {
16                 Console.WriteLine("Invalid directory path.");
17                 return;
18             }
19
20             // Get all files
21             string[] files = Directory.GetFiles(folderPath);
22
23             if (files.Length == 0)
24             {
25                 Console.WriteLine("No files found in the directory.");
26                 return;
27             }
28         }
29         catch { }
30     }
31 }
```

The screenshot shows a Visual Studio Code editor window with a C# file named Program.cs. The code implements a program that prompts the user for a folder path, validates its existence, and lists the files within it. The code is written in C# and uses the System and System.IO namespaces. It includes a try-catch block to handle potential exceptions. The Visual Studio Code interface is visible, showing the file explorer on the left and the code editor in the center. The Windows taskbar is visible at the bottom of the screen.



```
26         return;
27     }
28
29     Console.WriteLine("\nFiles in Directory:\n");
30
31     foreach (string file in files)
32     {
33         FileInfo fileInfo = new FileInfo(file);
34
35         Console.WriteLine("File Name: " + fileInfo.Name);
36         Console.WriteLine("File Size: " + fileInfo.Length + " bytes");
37         Console.WriteLine("Created On: " + fileInfo.CreationTime);
38         Console.WriteLine("-----");
39     }
40
41     Console.WriteLine("\nTotal Files: " + files.Length);
42
43     catch (UnauthorizedAccessException)
44     {
45         Console.WriteLine("Error: Access denied to the folder.");
46     }
47     catch (Exception ex)
48     {
49         Console.WriteLine("Unexpected error: " + ex.Message);
50     }
51
52     Console.ReadLine();
53 }
54
```

Output:



```
Enter folder path: C:\Users\gurun\OneDrive\Desktop
Files in Directory:
File Name: Batch7_Gurunath_Rageni_Employee_Management_System.zip
File Size: 13739765 bytes
Created On: 20-03-2026 16:34:08
-----
File Name: desktop.ini
File Size: 480 bytes
Created On: 27-11-2021 12:05:18
-----
File Name: Employee_Management_project - Shortcut.lnk
File Size: 1012 bytes
Created On: 17-03-2026 22:54:21
-----
File Name: File Explorer.lnk
File Size: 407 bytes
Created On: 03-12-2021 21:47:44
-----
File Name: GitHub.lnk
File Size: 2848 bytes
Created On: 13-09-2023 19:02:26
-----
File Name: Google Chrome.lnk
```

Problem 3 – Level 1:

Employee Performance Evaluator using Tuples and Pattern Matching (Level-1)

Scenario

A small company wants a simple program that evaluates an employee's performance based on **two inputs: monthly sales amount and customer feedback rating**.

The system should return both values together using a Tuple, and then determine the **performance category using pattern matching**.

This will help management quickly identify whether an employee is a **High Performer, Average Performer, or Needs Improvement**.

Requirements

1. Accept the following inputs from the user:
 - Employee Name
 - Monthly Sales Amount
 - Customer Feedback Rating (1–5)
2. Create a method that returns Sales Amount and Rating together using a Tuple.
3. Use pattern matching with switch expression or switch statement to categorize performance:
 - High Performer → Sales ≥ 100000 AND Rating ≥ 4
 - Average Performer → Sales ≥ 50000 AND Rating ≥ 3
 - Needs Improvement → All other cases
4. Display:
 - Employee Name
 - Sales Amount
 - Rating
 - Performance Category

Technical Constraints

- The method must return multiple values using Tuple.
- Pattern matching must be used for performance classification.
- Avoid using multiple nested if-else statements.
- Input values must be numeric and valid.

Expectations

- Students should demonstrate how to create and return tuples from a method.
- Students should use pattern matching to simplify conditional logic.

- Output should be clearly formatted.

Example Output:

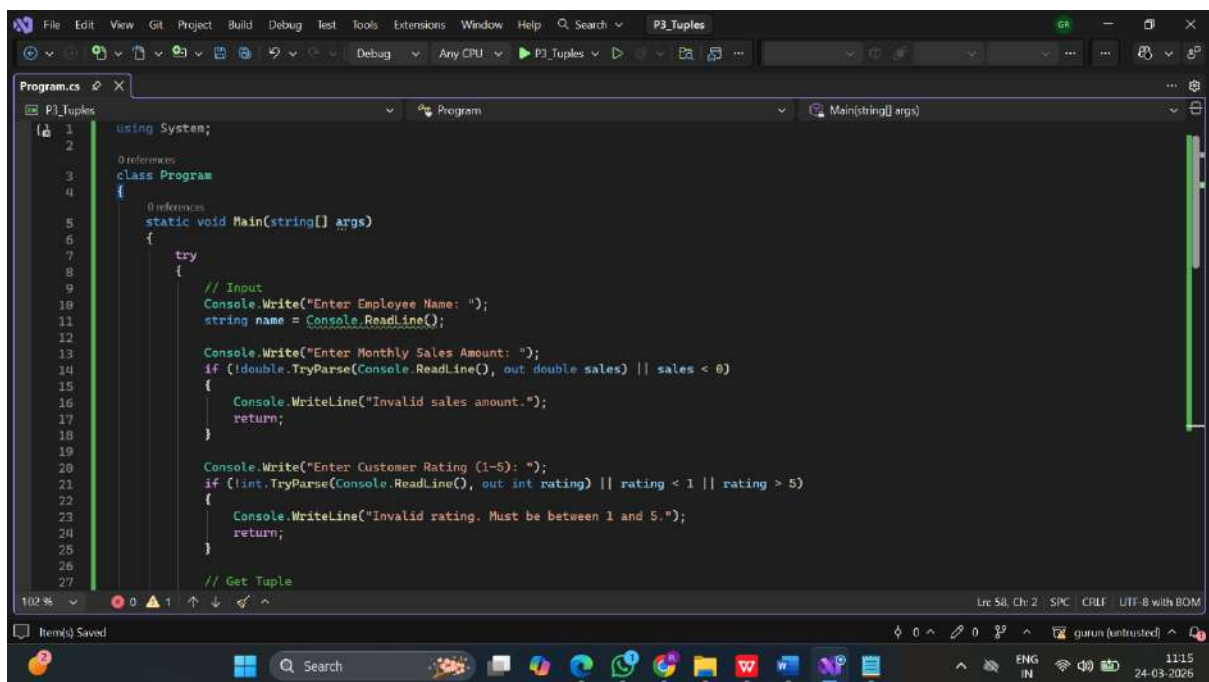
Employee Name: Rahul
Sales Amount: 120000
Rating: 4
Performance: High Performer

Learning Outcome

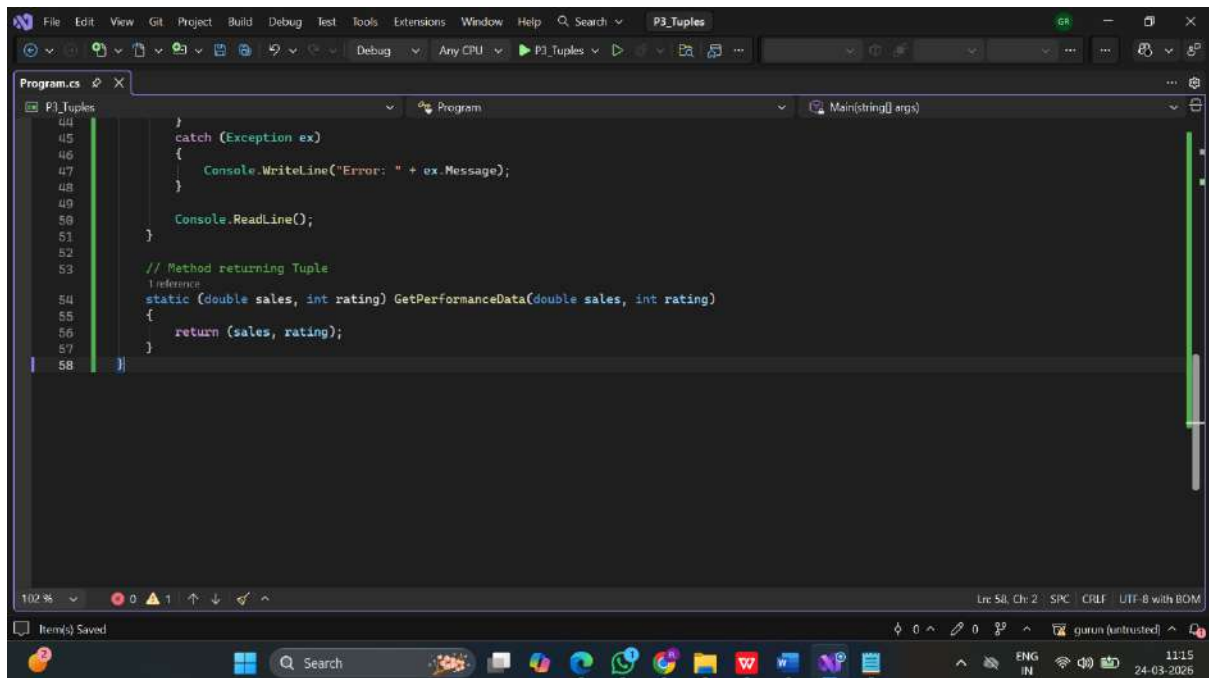
After completing this problem, students will be able to:

- Understand how tuples return multiple values from methods.
- Apply pattern matching to simplify complex conditions.
- Write cleaner and more readable decision logic in C#.

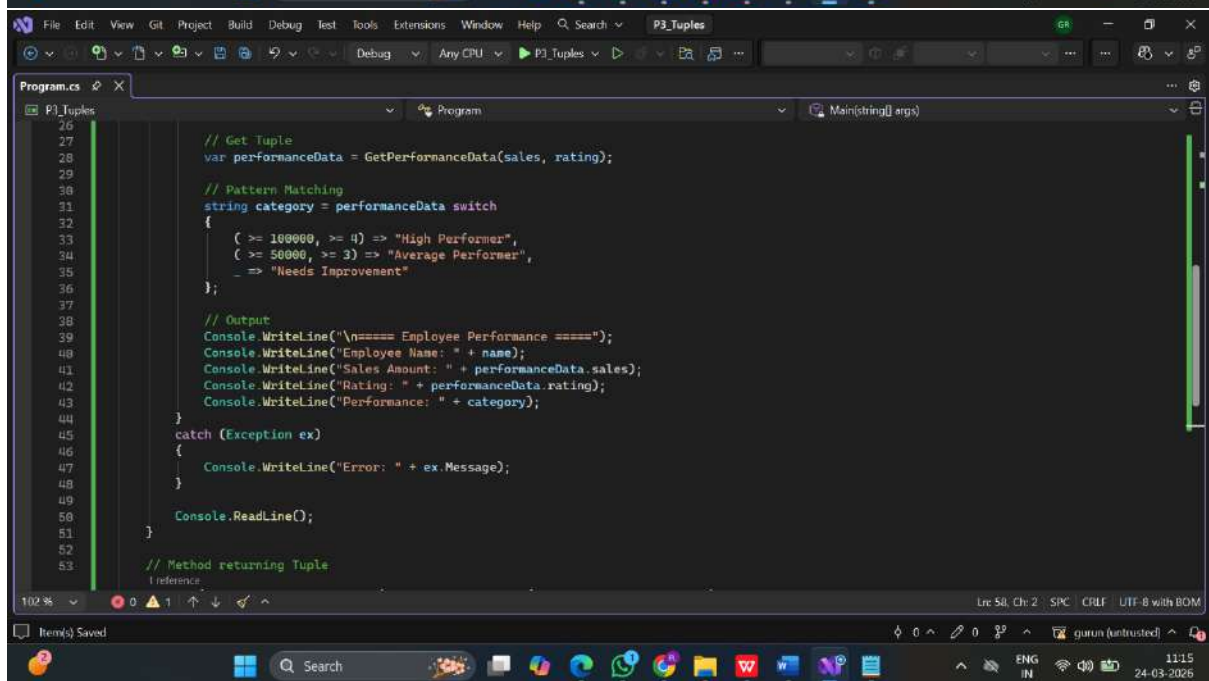
Code Screenshot:



```
Program.cs
1  using System;
2
3  class Program
4  {
5      static void Main(string[] args)
6      {
7          try
8          {
9              // Input
10             Console.WriteLine("Enter Employee Name: ");
11             string name = Console.ReadLine();
12
13             Console.WriteLine("Enter Monthly Sales Amount: ");
14             if (!double.TryParse(Console.ReadLine(), out double sales) || sales < 0)
15             {
16                 Console.WriteLine("Invalid sales amount.");
17                 return;
18             }
19
20             Console.WriteLine("Enter Customer Rating (1-5): ");
21             if (!int.TryParse(Console.ReadLine(), out int rating) || rating < 1 || rating > 5)
22             {
23                 Console.WriteLine("Invalid rating. Must be between 1 and 5.");
24                 return;
25             }
26
27             // Get Tuple
```

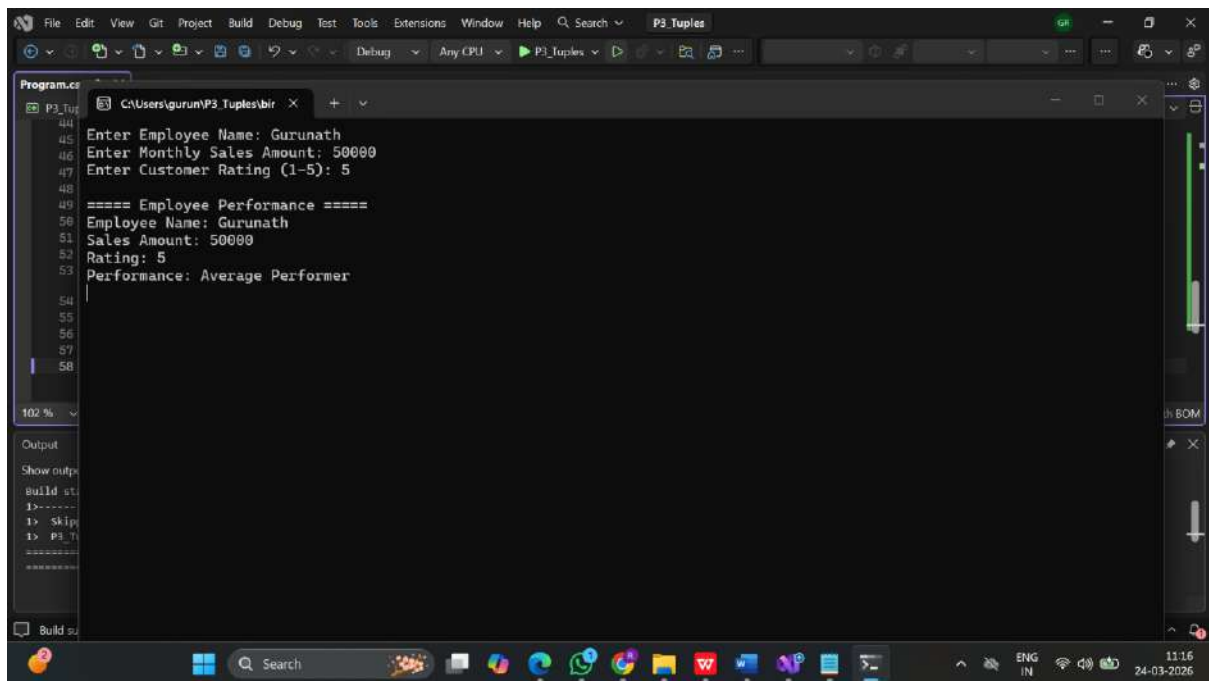


```
Program.cs
P3_Tuples
44
45
46     catch (Exception ex)
47     {
48         Console.WriteLine("Error: " + ex.Message);
49     }
50
51     Console.ReadLine();
52
53
54     // Method returning Tuple
55     static (double sales, int rating) GetPerformanceData(double sales, int rating)
56     {
57         return (sales, rating);
58     }
59 }
```



```
Program.cs
P3_Tuples
26
27
28     // Get Tuple
29     var performanceData = GetPerformanceData(sales, rating);
30
31     // Pattern Matching
32     string category = performanceData switch
33     {
34         ( >= 100000, >= 4) => "High Performer",
35         ( >= 50000, >= 3) => "Average Performer",
36         _ => "Needs Improvement"
37     };
38
39     // Output
40     Console.WriteLine("\n===== Employee Performance =====");
41     Console.WriteLine("Employee Name: " + name);
42     Console.WriteLine("Sales Amount: " + performanceData.sales);
43     Console.WriteLine("Rating: " + performanceData.rating);
44     Console.WriteLine("Performance: " + category);
45
46     catch (Exception ex)
47     {
48         Console.WriteLine("Error: " + ex.Message);
49     }
50
51     Console.ReadLine();
52
53
54     // Method returning Tuple
55     static (double sales, int rating) GetPerformanceData(double sales, int rating)
56     {
57         return (sales, rating);
58     }
59 }
```


Output:



```
Program.cs
C:\Users\gurun\P3_Tuples\bin\
44
45 Enter Employee Name: Gurunath
46 Enter Monthly Sales Amount: 50000
47 Enter Customer Rating (1-5): 5
48
49 ===== Employee Performance =====
50 Employee Name: Gurunath
51 Sales Amount: 50000
52 Rating: 5
53 Performance: Average Performer
54
55
56
57
58
102 %
Output
Show output
build st.
1>-----
1> Skip
1> P3.T
Build su
```

Problem 4 – Level 2

Scenario:

A development team wants to analyze all folders inside a project directory to understand the project structure.

Requirements:

1. Accept a root directory path.
2. Display all subdirectories inside the root folder.
3. Show the number of files present in each directory.

Technical Constraints:

- Use DirectoryInfo class.
- Use loops to iterate through directories.
- Implement exception handling.

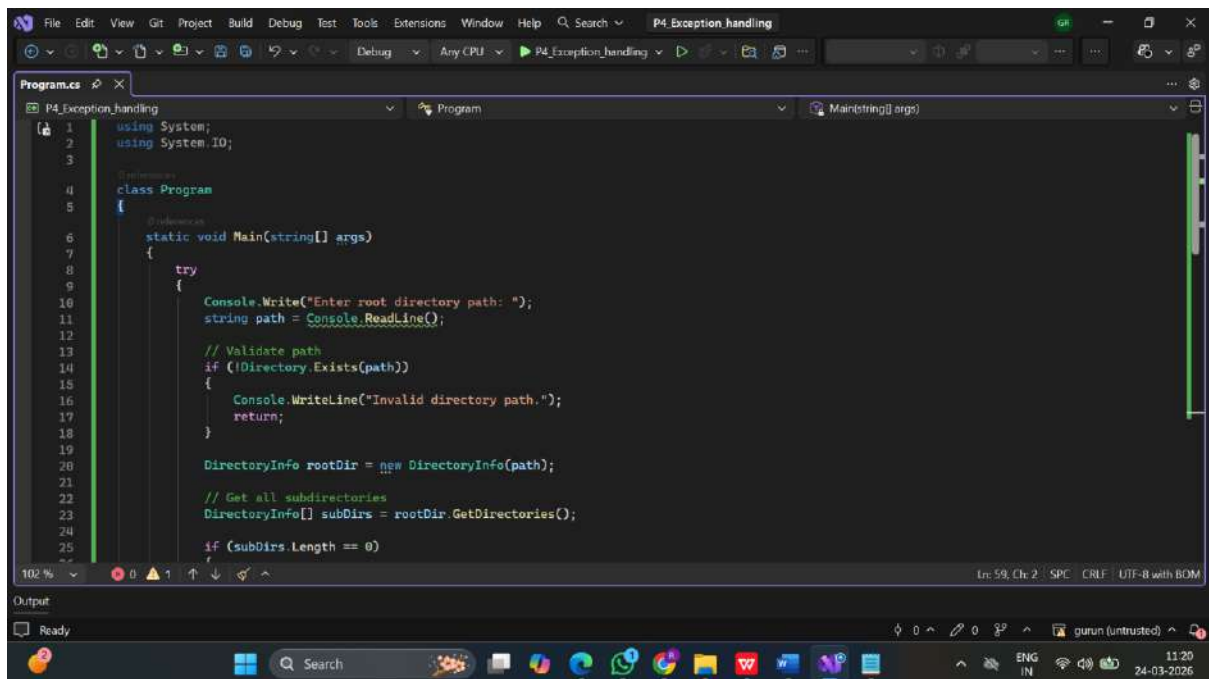
Expectations:

The application should display folder names and file counts for each directory.

Learning Outcome:

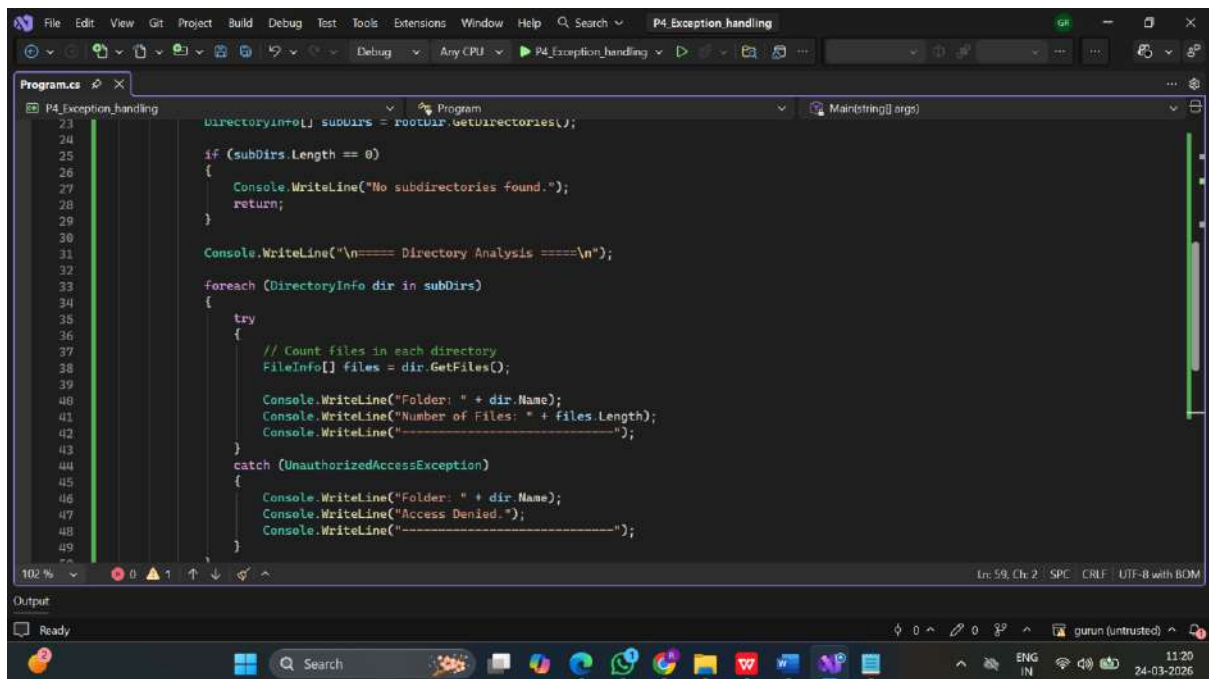
Students will learn how to navigate directories and retrieve folder information using DirectoryInfo.

Code Screenshots:



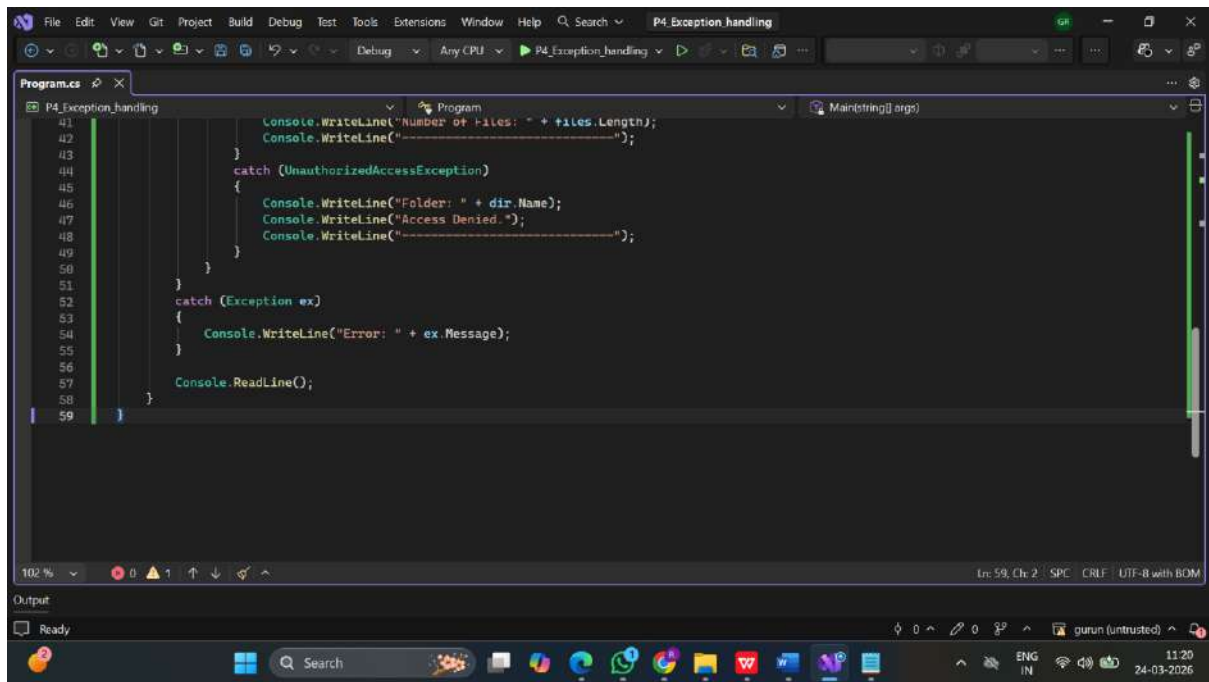
This screenshot shows the first part of a C# program in Visual Studio. The code defines a class `Program` with a static `Main` method. The method prompts the user to enter a root directory path, validates it, and then retrieves subdirectories. The code is as follows:

```
1 using System;
2 using System.IO;
3
4 class Program
5 {
6     // Main method
7     static void Main(string[] args)
8     {
9         try
10         {
11             Console.WriteLine("Enter root directory path: ");
12             string path = Console.ReadLine();
13
14             // Validate path
15             if (!Directory.Exists(path))
16             {
17                 Console.WriteLine("Invalid directory path.");
18                 return;
19             }
20
21             DirectoryInfo rootDir = new DirectoryInfo(path);
22
23             // Get all subdirectories
24             DirectoryInfo[] subDirs = rootDir.GetDirectories();
25
26             if (subDirs.Length == 0)
```



This screenshot shows the continuation of the C# program. It includes a `foreach` loop to iterate through subdirectories, a `try-catch` block to handle `UnauthorizedAccessException`, and logic to count files in each directory. The code is as follows:

```
23 DirectoryInfo[] subDirs = rootDir.GetDirectories();
24
25 if (subDirs.Length == 0)
26 {
27     Console.WriteLine("No subdirectories found.");
28     return;
29 }
30
31 Console.WriteLine("\n==== Directory Analysis =====\n");
32
33 foreach (DirectoryInfo dir in subDirs)
34 {
35     try
36     {
37         // Count files in each directory
38         FileInfo[] files = dir.GetFiles();
39
40         Console.WriteLine("Folder: " + dir.Name);
41         Console.WriteLine("Number of Files: " + files.Length);
42         Console.WriteLine("-----");
43     }
44     catch (UnauthorizedAccessException)
45     {
46         Console.WriteLine("Folder: " + dir.Name);
47         Console.WriteLine("Access Denied.");
48         Console.WriteLine("-----");
49     }
```

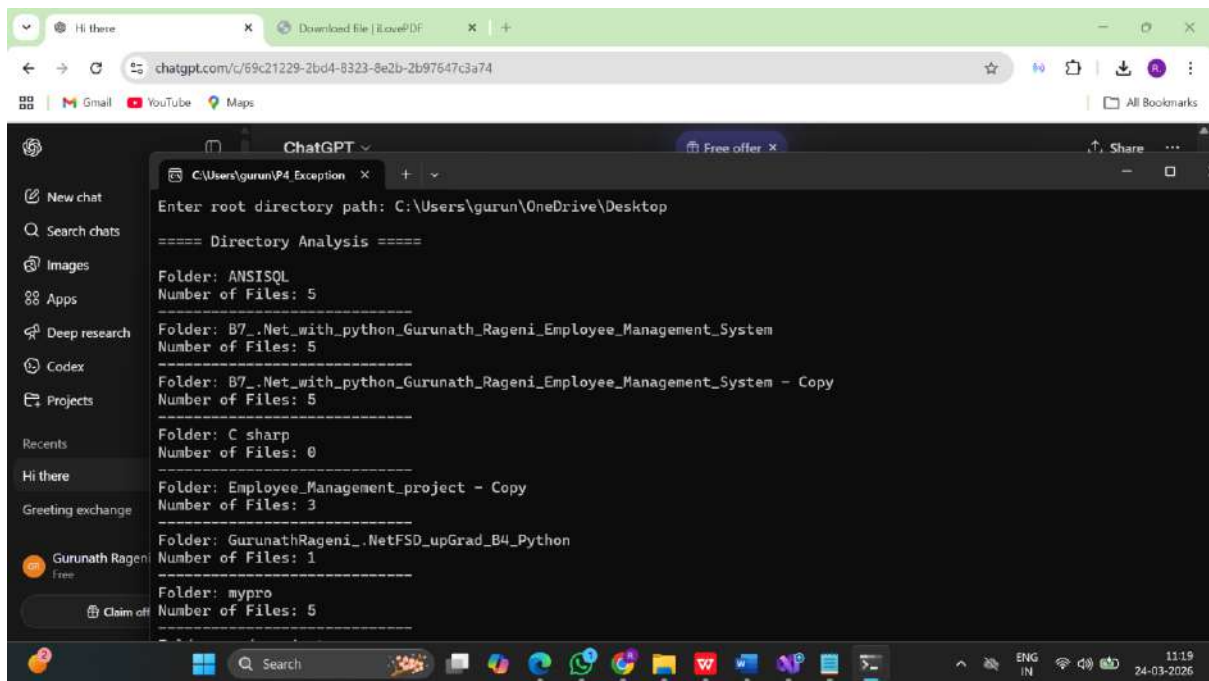


The screenshot shows the Visual Studio IDE with a C# file named `P4_Exception_handling.cs`. The code is as follows:

```
41 Console.WriteLine("Number of files: " + files.Length);
42 Console.WriteLine("-----");
43
44 catch (UnauthorizedAccessException)
45 {
46     Console.WriteLine("Folder: " + dir.Name);
47     Console.WriteLine("Access Denied.");
48     Console.WriteLine("-----");
49 }
50
51 }
52
53 catch (Exception ex)
54 {
55     Console.WriteLine("Error: " + ex.Message);
56 }
57
58 Console.ReadLine();
59 }
```

The bottom of the window shows the Output window with the text "Ready" and the Windows taskbar at the bottom.

Output:



The screenshot shows a web browser window with a ChatGPT chat interface. The user has entered the root directory path: `C:\Users\gurun\OneDrive\Desktop`. The chat response is as follows:

```
==== Directory Analysis ====

Folder: ANSISQL
Number of Files: 5
-----
Folder: B7_.Net_with_python_Gurunath_Rageni_Employee_Management_System
Number of Files: 5
-----
Folder: B7_.Net_with_python_Gurunath_Rageni_Employee_Management_System - Copy
Number of Files: 5
-----
Folder: C sharp
Number of Files: 0
-----
Folder: Employee_Management_project - Copy
Number of Files: 3
-----
Folder: GurunathRageni_.NetFSD_upGrad_B4_Python
Number of Files: 1
-----
Folder: mypro
Number of Files: 5
-----
```

The ChatGPT interface includes a sidebar with options like "New chat", "Search chats", "Images", "Apps", "Deep research", "Codex", "Projects", "Recents", and "Hi there". The user's name "Gurunath Rageni" is visible in the bottom left corner.